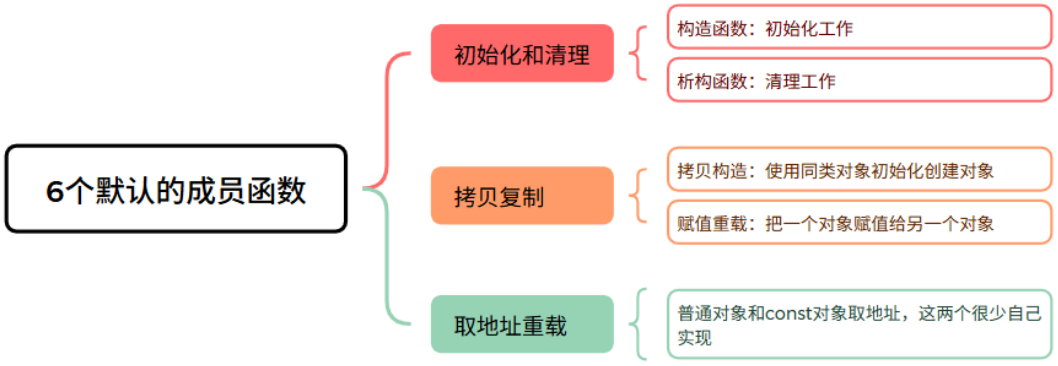
2026-4-22 C++类和对象：中

- 1. 类的默认成员函数
- 2.构造函数
  - 2.1构造函数的特点
- 3. 析构函数
- 3.1 析构函数的特点
- 4.拷贝构造函数
  - 4.1拷贝构造的特点
- 5.赋值运算符重载
  - 5.1运算符重载
  - 5.2赋值运算符重载
  - 5.2.1赋值运算符重载的意义
- 6. 取地址运算符重载
  - 6.1const成员函数
  - 6.2取地址运算符重载

1. 类的默认成员函数

什么是默认成员函数?
就是用户没有显示实现，但是编译器会自动生成的成员函数。
在一个类中，我们不写的情况下，编译器会默认生成6个成员函数，当然这六个成员函数最重要的是前三个，最后两个取地址重载相对没有那么重要，稍微了解即可。其次，C++11以后还会默认添加两个成员函数：移动构造和移动赋值。
我们将从以下两个方面展开学习：
第一：我们不写时，编译器默认生成的函数行为是什么？是否满足我们的需求?
第二：编译器默认生成的函数不满足我们的需求，我们需要自己实现，那应该如何实现?



2.构造函数

构造是特殊的成员函数，需要注意的是，构造函数虽然名称叫做构造，但是构造函数的主要任务并不是开空间创建对象（我们使用的局部对象是栈帧创建时，空间就开好了），而是对象实例化时初始化对象。构造函数的本质还是要替代我们之前的Stack和Date类中写的Init函数功能，构造函数自动调用的特点可以完美替代Init。

2.1构造函数的特点

1. 函数名与类名相同
2. 无返回值（没有返回值，且不需要写void,C++规定的)
3. 对象实例化时系统会自动调用对应的构造函数
4. 构造函数可以重载
5. 如果类中没有显示的定义构造函数，则C++编译器会自动生成一个无参的默认构造函数，一但用户默认定义，编译器将不再生成
6. 无参构造函数、全缺省构造函数、我们不写是编译器默认生成的构造函数都叫做默认构造函数（不要认为只有编译器生成的这个才叫默认构造，其实都是默认构造)。但是这三种构造函数只能有一个存在，不能同时存在。无参构造函数和全缺省构造函数最然都构成函数重载，但是调用时会存在歧义。**总结一下：就是不传递实参就可以调用的构造叫做默认构造**
7. 当我们不写的时候，编译器默认生成构造函数，对内置类型成员的初始化没有要求，也就是说能否初始化是不确定的，看编译器。对于自定义类型的成员变量，要求调用这个成员变量的默认构造函数初始化。如果这个成员变量，没有默认构造函数，就会报错，我么要初始化这个成员变量，需要用初始化列表才能解决。
   说明：C++把类型分为了内置类型（基本类型）和自定义类型。内置类型就是语言提供的原生数据类型，如：int/char/double等，自定义类型就是我们使用的class/struct等关键字自定义的类型。

```
#include<iostream>
using namespace std;
class Date
{
public:
    //1.无参构造函数
    Date( )
    {
        _year = 1;
        _month = 1;
        _day = 1;
    }
    //2.带参构造函数
```

```
    /*Date(int year, int month, int day)
    {
        _year = year;
        _month = month;
        _day = day;
    }*/
    //3.全缺省构造函数
    /*Date(int year = 1, int month = 1, int day =1)
    {
        _year = year;
        _month = month;
        _day = day;
    }*/
private:
    //如下示例
    int _year;
    int _month;
    int _day;
};
int main() {
    //如果只留下三个构造中的第二个带参构造
    //报错：没有合适的默认构造参数可以用
    //Date  d1;
    //Date d1(2022, 2, 3);
    //注意：如果通过无参构造创建对象时，对象后面不用跟括号
    //否则编译器无法区分这里是函数声明还是实例化对象
    Date d2;//注释1和3，不要括号
    return 0;
}
```

示例二：

```
class Stack {
public:
    Stack(int n = 4) {//带参全缺省构造
        array = (int*)malloc(sizeof(int) * n);
        if (nullptr == array) {
            perror("malloc申请空间失败");
            return;
        }
        capacity = n;
        top = 0;
    }
private://成员变量，一般不让修改
    int* array;//这里并不是变量的定义，这里是变量的声明
```

```
    size_t capacity;
    size_t top;
};
//两个队列实现栈
class MyQueue
{
public:
    //编译器默认生成MyQueue的构造函数调用了Stack的构造，完成了两个成员的初始化
private:
    Stack pushst;
    Stack popst;
};
int main() {
    MyQueue mq;//实例化一个对象，所以说st实际上就是我们的对象
    return 0;
}
```

3. 析构函数

析构函数的功能与构造函数的功能相反，析构函数不是完成对象本身的销毁，比如局部对象是存在栈帧的，函数结束了栈帧销毁，不用我们自己去销毁。C++规定对象在销毁的时候会调用析构函数，完成对象中资源的清理和释放工作。类比我们的Destroy功能，之前我们的Date没有Destroy,其实就是没有资源需要释放，所以严格说Date是不需要析构函数的。

3.1 析构函数的特点

1. 析构函数名称是在类名前加上字符~
2. **无参数返回值**，也不需要加（void)
3. 一个类只能有一个析构函数，若未显示定义，系统会自动生成默认的析构函数
4. 对象生命周期结束时，系统会自动调用析构函数
5. 和构造函数类似，我们不写，编译器自动生成的析构函数对内置类型成员不做处理，自定类型成员会调用他的析构函数。
6. 我们显示写析构函数，对于自定义类型成员也会调用他的析构，也就是说自定义类型成员无论什么情况都会自动调用析构函数
7. 如果类中没有申请资源的时候，析构函数可以不写，直接使用编译器默认的析构函数
8. 一个局部域的多个对象，C++规定后定义的先析构

```
class Stack {
public:
    Stack(int n = 4) {//带参全缺省构造
        array = (int*)malloc(sizeof(int) * n);
```

```
        if (nullptr == array) {
            perror("malloc申请空间失败");
            return;
        }
        capacity = n;
        top = 0;
    }
    ~Stack()//析构函数
    {
        cout << "~Stack()" << endl;//提示作用

        free(array);
        array = nullptr;
        top = capacity = 0;
    }
private://成员变量，一般不让修改
    int* array;//这里并不是变量的定义，这里是变量的声明
    size_t capacity;
    size_t top;
};

class MyQueue
{
public:
    //编译器默认生成MyQueue的析造函数调用了Stack的析构造，释放Stack内部的资源
    //显示写析构函数，也会自动调用Stack的析构
    /*~MyQueue();*/
private:
    Stack pushst;
    Stack popst;
};
int main() {
    MyQueue mq;//实例化一个对象，所以说st实际上就是我们的对象
    return 0;
}
```

示例二：C++实现括号匹配问题
我们可以发现三个相较于C语言比较便利的点：

1. 不需要初始化栈
2. 不需要将栈的地址作为参数传递到Push,Pop,Empty作为参数传递进去
3. 不用怕忘记销毁栈，很好的防止了内存泄露

```
bool isValid(const char* s) {
    Stack st;
    while (*s) {
        if (*s == '[' || *s == '(' || *s == '{')
        {
            st.Push(*s);
        }
        else {
            if (st.Empty()) {
                return false;
            }
            char top = st.Top();
            st.Pop();
            if ((*s == ']' && top != '[') || (*s == ')' && top != '(') ||
(*s == '}' && top != '{'))
            {
                return false;
            }
        }
        ++s;
    }
    return st.Empty();
}
int main() {
    cout << isValid("[()][]") << endl;
    return 0;
}
```

4.拷贝构造函数

如果一个构造函数的第一个参数是自身类类型的引用，且任何额外参数都有默认值，则此构造函数也叫拷贝函数，也就是说拷贝函数是一个特殊的构造函数。

4.1拷贝构造的特点

1. 拷贝构造是构造函数的一个重载
2. **拷贝构造的第一个参数必须是类类型对象的引用，使用传值方式编译直接报错，因为语法逻辑上会引发无穷递归调用**。拷贝构造函数也可以多个参数，但是第一个参数必须是类类型的引用，后面的参数必须有缺省值。
3. C++规定自定义类型对象进行拷贝行为必须调用拷贝函数，所以这里自定义类型传值传参和传值返回都会调用拷贝构造完成，所以这里自定义类型的传值传参和返回值都会调用拷贝函数构造完成。

4. 若未显示定义拷贝构造，编译器会生成拷贝的构造函数。自动生成的拷贝构造对内置类型成员变量会完成值拷贝/浅拷贝（一个字节一个字节的拷贝），对自定类型成员变量会调用其他的拷贝构造。
5. 像Date这样的类成员变量全是内置类型，且并没有什么指向的资源，编译器自动生成拷贝构造就可以完成需要的拷贝，所以不需要我们显示实现拷贝构造。像Stack这样的类，虽然也都是内置类型，但是array指向了资源，编译器自动生成的拷贝构造完成的值拷贝/浅拷贝不符合我们的需求，所以我们自己实现深拷贝（对指向的资源也要进行拷贝）。像MyQueue这样的类型内部主要是自定义类型的Stack成员，这个编译器在默认生成拷贝函数的时候会调用Stack的拷贝构造函数，所以也不需要我们显示实现。
6. 传值返回会产生一个临时对象调用拷贝构造，传值引用返回，返回的是返回对象的别名（引用），没有产生拷贝。但是如果返回对象是一个当前函数局部域的局部对象，函数结束就销毁了，那么使用引用返回是有问题的，这时的引用类似于一个野指针。穿引用返回可以减少拷贝，但是一定要确保返回对象，在当前函数结束后还在，才能引用返回。
   示例1：

```
class Date
{
public:
    Date(int year=1, int month=1, int day=1)
    {
        _year = year;
        _month = month;
        _day = day;
    }

    //拷贝构造函数
    Date(const Date& d) {
        _year = d._year;
        _month =d. _month;
        _day = d._day;
    }
    //拷贝构造函数2我们说指针和引用的底层具有一定的相似性
    Date(Date* d) {
        _year = d->_year;
        _month = d->_month;
        _day = d->_day;
    }
    void Print() {
        cout << _year << "_" << _month << "_" << _day << endl;
    }
private:
    //如下示例
    int _year;
    int _month;
```

```
    int _day;
};
void Func1(Date d) {
    cout << &d << endl;
    d.Print();
}
Date& Func2( ) {
    Date tmp(2026, 4, 23);
    tmp.Print();
    return tmp;
}
int main() {
    Date d1(2026,6,8);
    //C++规定自定义类型对象进行拷贝行为必须调用拷贝构造函数，所以这里传值传参需要调用拷贝构造
    //这里d1传值传参给d要调用拷贝构造完成拷贝，引用传参可以减少这里的拷贝
    Func1(d1);
    cout << &d1 << endl;
    //这里可以完成拷贝，但是不是拷贝构造，只是一个普通构造
    Date d2(&d1);
    d1.Print();
    d2.Print();

    //这样写才是拷贝构造，通过同类型的对象初始化构造，而不是指针
    Date d3(d1);
    d3.Print();
    //也可以这样写，这个也是拷贝构造
    Date d4 = d1;
    d4.Print();

    //Func2返回了一个局部变量tmp的应用作为返回值
    //Func2函数结束，tmp对象就销毁了，就相当于一个野引用
    Date ret = Func2();
    ret.Print();

    return 0;
}
```

示例2：

```
typedef int STDatatype;
class Stack {
public:
    Stack(int n = 4) {//带参全缺省构造
        array = (STDatatype*)malloc(sizeof(STDatatype) * n);
```

```
        if (nullptr == array) {
            perror("malloc申请空间失败");
            return;
        }
        capacity = n;
        top = 0;
    }
    Stack(const Stack& st) {//定义拷贝函数
        array = (STDatatype*)malloc(sizeof(STDatatype) * st.capacity);
        if (nullptr == array) {
            perror("malloc申请空间失败");
            return;
        }
        memcpy(array, st.array, sizeof(STDatatype) * st.top);
        capacity = st.capacity;
        top = st.top;
    }
    void Push(STDatatype x) {
        if (top == capacity) {
            int newcapacity = capacity * 2;
            STDatatype* tmp = (STDatatype*)realloc(array, newcapacity *
sizeof(STDatatype));
            if (tmp == NULL) {
                perror("fail");
                return;
            }
            array = tmp;
            capacity = newcapacity;
        }
        array[top++] = x;
    }
private://成员变量，一般不让修改
    int* array;//这里并不是变量的定义，这里是变量的声明
    size_t capacity;
    size_t top;
};
//两个队列实现栈
class MyQueue
{
public:
    //编译器默认生成MyQueue的构造函数调用了Stack的构造，完成了两个成员的初始化
private:
    Stack pushst;
    Stack popst;
};
int main() {
```

```
    Stack st1;
    st1.Push(1);
    st1.Push(2);
    //Stack不显示实现拷贝构造，用自动生成的拷贝构造完成浅拷贝
    //会导致st1和st2里面的，array指针指向同一块资源，析构的时候会析构两次
    Stack st2 = st1;
    MyQueue mq1;
    //MyQueue自动生成的拷贝构造，自动调用调用Stack拷贝构造完成pushst/popst
    //Stack拷贝构造自己实现了深拷贝，他就没有问题
    MyQueue mq2 = mq1;

    return 0;
}
```

5.赋值运算符重载

5.1运算符重载

1. 当运算符被用于类类型的对象时，C++允许我们通过运算符号重载的形式指定新的含义。C++规定类类型对象中使用运算符的时候，必须转换成调用对应的运算符重载，若没有对应的运算符重载，编译器报错。
2. 运算符重载是具有特殊名字的函数，他由operator和后面要定义的运算符共同构成。和其他函数一样，它也具有其返回类型和参数列表以及函数体。
3. 运算符函数的参数个数和该运算符作用的运算对象数量一样多。一元运算符有一个参数，二元运算符有两个参数，二元运算符的左侧运算对象传给第一个参数，右侧对象传递给第二个参数。
4. 如果一个重载运算符函数是成员函数，则它的第一个运算对象默认传给隐式的this指针，因此运算符作为成员函数时，参数比运算对象少一个。
5. 运算符重载以后，其优先级于结合性与对应内置类型的运算符保持一致。
6. 不能通过连接语法中没有的符号来创建运算符：比如operator@。
7. 不能重载的运算符“.、::、sizeof、？：、.”

```
class A
{
public:
    void func() {
        cout << "A::func()" << endl;
    }
};
typedef void(A::* PF)();//成员函数指针类型
int main() {
```

```
    //C++规定成员函数要加&才有&才能取到函数指针
    PF pf = &A::func;
    A obj;
    (obj.*pf)();
    return 0;
}
```

1. 重载操作符至少有一个类型的参数，不能通过运算符重载来改变内置类型对象的含义。
   如：int operator+(int x,int y);
2. 一个类需要重载那些运算符号是看那些运算符重载后有意义
3. 重载++运算符号时，有前置++和后置++，运算符重载函数名都是operator++,无法很好的区分，C++规定，后置++重载时，增加一个int形参，跟前置++构成函数重载，方便区分。
4. 重载<<和>>时，需要重载未=为全局函数，因为重载为成员函数，this指针默认抢占第一个形参的位置，第一个形参的位置是左侧运算对象，调用时就变成了对象<<cout不符合使用习惯和可读性。重载为全局函数把ostream/istream放到第一个形参的位置上就可以了，第二个形参当类类型对象。
   示例1：

```
class Date
{
public:
    Date(int year=1, int month=1, int day=1)
    {
        _year = year;
        _month = month;
        _day = day;
    }

    //拷贝构造函数
    Date(const Date& d) {
        _year = d._year;
        _month =d. _month;
        _day = d._day;
    }
    //拷贝构造函数2我们说指针和引用的底层具有一定的相似性
    Date(Date* d) {
        _year = d->_year;
        _month = d->_month;
        _day = d->_day;
    }
    void Print() {
        cout << _year << "_" << _month << "_" << _day << endl;
    }
```

```
private:
    //如下示例
    int _year;
    int _month;
    int _day;
};
void Func1(Date d) {
    cout << &d << endl;
    d.Print();
}
Date& Func2( ) {
    Date tmp(2026, 4, 23);
    tmp.Print();
    return tmp;
}
int main() {
    Date d1(2026,6,8);
    //C++规定自定义类型对象进行拷贝行为必须调用拷贝构造函数，所以这里传值传参需要调用拷贝构造
    //这里d1传值传参给d要调用拷贝构造完成拷贝，引用传参可以减少这里的拷贝
    Func1(d1);
    cout << &d1 << endl;
    //这里可以完成拷贝，但是不是拷贝构造，只是一个普通构造
    Date d2(&d1);
    d1.Print();
    d2.Print();

    //这样写才是拷贝构造，通过同类型的对象初始化构造，而不是指针
    Date d3(&d1);
    d3.Print();
    //也可以这样写，这个也是拷贝构造
    Date d4 = d1;
    d4.Print();

    //Func2返回了一个局部变量tmp的应用作为返回值
    //Func2函数结束，tmp对象就销毁了，就相当于一个野引用
    Date ret = Func2();
    ret.Print();

    return 0;
}
```

示例2：

```
class Date
{
public:
    Date(int year = 1, int month = 1, int day = 1)
    {
        _year = year;
        _month = month;
        _day = day;
    }
    void Print() {
        cout << _year << "_" << _month << "_" << _day << endl;
    }
    bool operator==(const Date& d) {//d1传递进去给上面哪个函数，d2传递进去给下面这
个
        return _year == d._year &&//d2所对应的值与d1所对应的值做对比
               _month == d._month &&
               _day == d._day;
    }
    Date& operator++() {
        cout << "前置++" << endl;
        _day = ++_day;
        return *this;
    }
    Date& operator++(int){
        Date tmp;
        cout << "后置++" << endl;
        _day = _day++;
        return tmp;
        }
private:
    //如下示例
    int _year;
    int _month;
    int _day;
};

int main() {
    Date d1(2026, 4, 23);
    Date d2(2026, 4, 4);
    //运算符重载函数可以显示调用
    d1.operator ==(d2);
    //这种写法也是支持的，编译器会转换成d1.operator ==(d2);
    d1 == d2;
    //编译器会自动转化为d1.operator++( );
    //++d2;
    //编译器会自动转化为d1.operator++(int);
```

```
    d2.Print();
    d2++;
    d2.Print();
    return 0;
}
```

5.2赋值运算符重载

用于完成两个**已经存在的对象**（根本原因）直接的拷贝赋值，这里要注意跟拷贝构造区分，拷贝构造用与一个对象拷贝初始化给另一个要创建的对象。
一个已经存在的对象去初始化另一个要实例化的对象

5.2.1赋值运算符重载的意义

1. 赋值运算符重载是一个运算符重载，规定必须重载为成员函数。赋值运算符重载的参数建议写成const当前类类型引用，否则传值传参会有拷贝。
2. 有返回值，且建议写成当前类类型的引用，引用返回可以提高效率，有返回值目的是为了支持连续赋值的场景。
3. 没有显示实现时候，编译器会自动生成一个默认赋值运算符重载函数，用于完成内置成员变量的浅拷贝，对自定义成员变量会调用他的赋值重载函数。（大致上都与

```
//赋值运算符重载
class Date
{
public:
    Date(int year = 1, int month = 1, int day = 1)
    {
        _year = year;
        _month = month;
        _day = day;
    }
    void Print() {
        cout << _year << "_" << _month << "_" << _day << endl;
    }
    Date(const Date& d) {//这个是拷贝构造
        cout << "Date(const Date& d)" << endl;
        _year = d._year;
        _month = d._month;
        _day = d._day;
    }
    Date& operator=(const Date& d) {//完成赋值运算符重载
        //不要检查自己给自己赋值的情况
```

```
        if (this != &d) {
            _year = d._year;
            _month = d._month;
            _day = d._day;
        }
        //若d1=d2表达式的返回值应该为d1,也就是*this
        return *this;
    }
private:
    //如下示例
    int _year;
    int _month;
    int _day;
};

int main() {
    Date d1(2026, 4, 23);
    Date d2(d1);//拷贝构造
    Date d3(2026, 4, 24);
    d1 = d3;//两个已有成员之间--赋值重载

    //这个是拷贝构造
    //赋值重载完成在两个已有成员之间
    //拷贝构造适用于一个对象拷贝初始化给另一个要创建的对象
    Date d4 = d1;
    return 0;
}
```

6. 取地址运算符重载

6.1const成员函数

```
#include<iostream>
using namespace std;
class Date
{
public:
    Date(int year = 1, int month = 1, int day = 1) {
        _year = year;
        _month = month;
        _day = day;
    }
    //这个东西就相当于:
    //void Print(const Date* const this) const既不能修改this指针的值
```

```
    // 也不能修改this指针的地址
    void Print() const//const修饰成员函数，const修饰成员函数放到成员函数参数列表后面
    {
        cout << _year << "-" << _month << "-" << _day << endl;
    }
private:
    int _year;
    int _month;
    int _day;
};
int main() {
    //这里非const对象也可以调用const成员函数是一种权限缩小
    Date d1(2024, 7, 5);
    d1.Print();

    const Date d2(2024, 8, 5);
    d2.Print();

    return 0;
}
```

1. 将const修饰的成员函数称为const成员函数，const修饰成员函数放到成员函数列表之后
2. const实际修饰的是该成员函数的隐含this指针，标明在该成员函数中不能改变其中的任何一个成员。const修饰Date类的Print成员函数，Print隐含的this指针由Date *const this*变为 *const Date* const this。解释一下就是不可改变地址变成了既不可以改变地址也不可以改变值。

6.2取地址运算符重载

分为两种：

1. 普通取地址重载
2. const取地址重载
   一般这两个编译器自动生成的就足够我们使用了，不需要我们显示实现。除非：我们不想让别人取到当前类对象的地址，就可以自己实现一份，胡乱返回一个地址。

```
class Date
{
public:
    Date* operator&()
    {
```

```
        return this;
        //return nullptr;
    }
    const Date* operator& ()const
    {
        return this;
    }
private:
    int _year;
    int _month;
    int _day;
};
```